Smart City Data Platform

Complete Technical Documentation

Cairo & New Administrative Capital, Egypt

ITI Graduation Project — Data Engineering Track

TABLE OF CONTENTS

---

1. PROJECT OVERVIEW

Project Name: Smart City Data Platform **Domain:** IoT, Real-Time Streaming, Batch Analytics, Smart Transportation **Location Focus:** Cairo + New Administrative Capital, Egypt **Team Size:** 5 members **Duration:** 1 month **Track:** Big Data Engineering — ITI (Information Technology Institute)

The Smart City Data Platform is a production-grade big data engineering system that ingests, processes, stores, and analyzes real-time vehicle telemetry data from 5 simulated vehicles operating across 4 real Cairo GPS routes. The platform demonstrates a complete modern data engineering stack covering real-time streaming (sub-second latency), batch processing (hourly aggregations), a

medallion data lake (Bronze/Silver/Gold on AWS S3), a dimensional data warehouse (Snowflake), a semantic transformation layer (dbt), and dual visualization paths (Grafana for real-time, Power BI for historical batch analytics).

---

2. BUSINESS PROBLEM & OBJECTIVES

Problem Statement

Cairo is one of the most congested cities in the world. The New Administrative Capital aims to be a smart city from inception. Both cities need:

- Real-time visibility into vehicle fleet performance
- Predictive fuel consumption analysis to reduce operational costs
- Incident detection and safety monitoring
- Traffic congestion analysis for urban planning
- Environmental impact tracking (CO2 emissions)

Objectives

1. Build a real-time pipeline that detects vehicle anomalies within 5 seconds
2. Create a historical analytics warehouse covering 30+ days of fleet data
3. Enable cross-topic analysis: how does Cairo weather affect fuel consumption?
4. Demonstrate Medallion Architecture (Bronze/Silver/Gold) on AWS S3
5. Apply dimensional modeling (Galaxy Schema) in Snowflake
6. Implement data quality testing (64 automated dbt tests — all passing)
7. Provide a foundation for future ML-based predictive maintenance

---

3. SYSTEM ARCHITECTURE

Two Data Paths

Real-Time Path (sub-second latency):

```
Vehicle Simulator (Python)
    → Kafka topic: vehicle-telemetry (3 partitions, 5 msg/sec)
    → Python Alert Consumer (threshold checks, 1-sec response)
    → Kafka topic: alerts + PostgreSQL realtime_alerts table
    → Grafana Dashboard (auto-refresh 15s)
```

Batch Path (hourly):

```
Vehicle Simulator (Python)
    → Kafka (5 topics: telemetry, weather, traffic, road-events, alerts)
    → Spark Bronze Writer (30s micro-batch) → S3 Bronze (raw Parquet)
    → Spark Silver Cleaner (60s trigger) → S3 Silver (cleaned Parquet)
    → Airflow @hourly → Spark Gold Aggregator (batch) → S3 Gold (KPI Parquet)
    → Snowflake External Tables (RAW schema — zero data movement)
```

```
    → dbt models (STAGING views + MART tables)
    → Power BI (Import mode, manual refresh)
    → ML Path (S3 Gold Parquet → future SageMaker/model training)
```

Why Two Paths?

Operations teams need real-time alerts (seconds). Management teams need historical trends (days/weeks). These are different use cases requiring different tools. Grafana excels at time-series monitoring. Power BI excels at business intelligence and dimensional analysis.

---

4. DATA SOURCES & SIMULATION

Physics-Based Vehicle Simulator

File: `simulator/vehicle_simulator.py`

The simulator generates **correlated, realistic** vehicle data — not random numbers. Variables are causally linked to match real automotive engineering:

Correlation chain:

```
Road Type → Base Speed
Traffic Density → Speed Factor → Actual Speed
Weather Condition → Speed Multiplier → Actual Speed
Actual Speed → RPM (via gear ratio + tire circumference formula)
RPM → Fuel Rate (base + RPM penalty + idle penalty + A/C load)
Engine Temp → Cold Start enrichment (extra fuel while warming up)
```

Formula for RPM:

```
RPM = (speed m/s / tire circumference m) × gear ratio × final drive × 60
```

Formula for Fuel Rate:

```
fuel rate = base l100km + rpm penalty + idle penalty + ac load + cold penalty
```

This means: if you see high fuel consumption in the data, it will always be accompanied by either high RPM, idling (speed < 5), high temperature (A/C), or a cold engine — just like a real vehicle.

4 Real Cairo Routes (OpenStreetMap GPS coordinates)

Route	Distance	Road Types	Key Points
tahrir_to_new_capital	~60 km	urban → arterial → highway	Ring Road East → Cairo-Suez
maadi_to_nasr_city	~22 km	arterial → highway	Ring Road → Autostrad
giza_to_heliopolis	~30 km	urban → arterial	6th October Bridge → Airport
sixth_october_to_maadi	~38 km	highway → arterial	Desert Road → Giza

5 Vehicle Types

Type	Fuel Base	Tank	Real-World Equivalent
sedan	8.0 L/100km	55 L	Toyota Corolla / Hyundai Elantra
suv	11.0 L/100km	70 L	Toyota RAV4 / Hyundai Tucson
microbus	12.0 L/100km	70 L	Toyota Hiace (common in Cairo)

External APIs

- **OpenWeatherMap** — real Cairo weather (5-min cache, free tier: 1,000 calls/day)
- **TomTom Traffic API** — real congestion data (60-sec cache, free tier: 2,500 calls/day)
- **Fallback:** Cairo seasonal weather model + rush-hour simulation model (no API key needed — works offline)

Cairo Rush-Hour Model (traffic_fetcher.py)

The simulator models Cairo's specific traffic patterns:

- Morning peak: 08:00–10:30 AM (speed factor: 0.42)
- Evening peak: 16:00–19:30 PM (speed factor: 0.38 — heavier than morning)
- Friday: reduced but afternoon family traffic (14:00–18:00)
- Saturday: Cairo weekend, lighter traffic
- Downtown (Tahrir area): zone_factor = 1.3 (30% more congested)
- Desert highways: zone_factor = 0.65 (65% of downtown congestion)

---

5. INFRASTRUCTURE LAYER (Docker)

File: `docker-compose.yml` **Total services:** 13 containers **Total memory requirement:** ~8 GB minimum, 16 GB recommended

Service Inventory

Container	Image	Port	Purpose
sc_zookeeper	confluentinc/cp-zookeeper:7.5.0	2181	Kafka coordination
sc_kafka	confluentinc/cp-kafka:7.5.0	9092, 9101	Message broker
sc_kafka_init	confluentinc/cp-kafka:7.5.0	—	Creates 5 topics on startup
sc_kafka_ui	provectuslabs/kafka-ui	8080	Kafka browser UI

Container	Image	Port	Purpose
sc_spark_master	apache/spark:3.5.3	7077, 8081	Spark cluster master
sc_spark_worker_1	apache/spark:3.5.3	—	Executor (2 cores, 2000 MiB)
sc_spark_worker_2	apache/spark:3.5.3	—	Executor (2 cores, 2000 MiB)
sc_spark_worker_3	apache/spark:3.5.3	—	Executor (2 cores, 2000 MiB)
sc_postgres	postgres:15-alpine	5432	Airflow metadata + alert storage
sc_airflow	apache/airflow:2.8.1	8082	Workflow orchestration
sc_prometheus	prom/prometheus:v2.48.0	9090	Metrics collection
sc_grafana	grafana/grafana:10.2.2	3000	Real-time dashboards
sc_simulator	custom Python image	—	Vehicle data generator

Kafka Dual-Listener Design

Kafka needs two listeners because containers and the host machine use different addresses:

- `INTERNAL://kafka:29092` — used by Spark containers (Docker network)
- `PLAINTEXT://localhost:9092` — used by Python scripts on host machine

Spark Resource Management

Problem encountered: With 6 total cores (3 workers × 2 cores), Bronze grabbed all cores and Silver starved. Solution: explicit `--conf spark.cores.max` limits:

- Bronze Writer: 2 cores, 512 MiB executor memory
- Silver Cleaner: 1 core, 512 MiB executor memory
- Gold Aggregator: 1 core, 512 MiB executor memory (batch via Airflow)
- Alert Consumer: 0 Spark cores (pure Python on Windows host)

---

6. INGESTION LAYER (Apache Kafka)

5 Kafka Topics

Topic	Partitions	Retention	Rate	Purpose
vehicle-telemetry	3	24h	5 msg/sec	Main telemetry from 5 vehicles
weather-data	1	24h	1 msg/5min	Cairo weather snapshots
traffic-events	2	24h	5 msg/min	TomTom

Topic	Partitions	Retention	Rate	Purpose
road-events	1	48h	~1% of ticks	ACCIDENT/ROADWORK/BREAKDOWN/CONGESTION
alerts	1	7d	On anomaly	Real-time threshold violations

Why Kafka?

1. **Decoupling:** Simulator and Spark are completely independent. Either can restart without affecting the other.
2. **Durability:** Messages persist for 24-48 hours — Spark can replay from any offset.
3. **Scalability:** vehicle-telemetry has 3 partitions — can scale to 3 parallel Spark tasks reading simultaneously.
4. **Back-pressure:** `maxOffsetsPerTrigger=5000` limits how fast Spark reads, preventing memory overflow.

Alert Thresholds (config.py)

Metric	Threshold	Severity	Real-World Meaning
speed_kmh	> 120 km/h	HIGH	Exceeds Egyptian highway limit
engine_temp_c	> 105°C	CRITICAL	Imminent overheating, stop vehicle
fuel_level_pct	< 10%	MEDIUM	Less than ~5L remaining
rpm	> 5000	HIGH	Engine stress, gear change needed
acceleration_ms2	< -4.0	HIGH	Hard braking / collision indicator
idle + temp > 98°C	combined	HIGH	Stuck in traffic, cooling failure

---

7. PROCESSING LAYER (Apache Spark)

Why Spark?

Kafka produces 5 messages/second = 18,000 messages/hour = 432,000 messages/day. Processing each one individually is inefficient. Spark reads in micro-batches, applies transformations in parallel across 3 workers, and writes efficiently to S3 using columnar Parquet format.

bronze_writer.py — Kafka → S3 Bronze

- Reads all 4 producer topics simultaneously (4 parallel streaming queries)
- Adds Kafka metadata: topic, partition, offset, kafka_timestamp
- Writes raw Parquet every 30 seconds (micro-batch trigger)
- Uses S3A multipart upload for efficiency
- Checkpoint stored in S3 `_checkpoints/` folder (never delete this)

Key design: No transformation at this layer. Raw = raw. If Silver has a bug, Bronze allows complete reprocessing from scratch.

silver_cleaner.py — S3 Bronze → S3 Silver

- Reads Bronze Parquet as streaming source (4 independent queries)
- Applies data quality rules:
    - Drop rows missing mandatory fields (event_id, vehicle_id, timestamp_unix)
    - Clamp sensor values to physical limits:
        - speed_kmh: 0–250 km/h
        - rpm: 0–8,000
        - engine_temp_c: 15–120°C
        - fuel_level_pct: 0–100%
        - latitude: 29.5–30.5 (Cairo bounding box)
        - longitude: 30.5–32.0 (Cairo bounding box)
- Derives categorical columns:
    - speed_band: stopped/slow/medium/fast/overspeed
    - fuel_band: critical/low/ok/full
    - is_anomaly: boolean flag
    - weather_severity: low/moderate/high/severe
    - congestion_band: free_flow/light/moderate/heavy/gridlock
- Triggers every 60 seconds

Critical bug fixed: Bronze writes `kafka_timestamp` as `TimestampType` but original Silver schema declared it as `LongType`. This caused all rows to be null and dropped. Fixed by correcting the schema definition.

gold_aggregator.py v2 — S3 Silver → S3 Gold

Reads from ALL 4 Silver tables and produces 7 Gold aggregation tables:

From silver/telemetry:

1. `vehicle_5min` — 5-min window per vehicle: avg/max/min speed, RPM, engine

temp, fuel level, fuel rate, traffic density, trip distance, GPS, anomaly count

2. `route_hourly` — 1-hour window per route: speed bands, throughput, congestion, fuel rate, anomaly rate
3. `fuel_daily` — 1-day window per vehicle type: total fuel, CO2 estimate (2.31 kg/litre for Egypt 95-octane), cost estimate (11 EGP/litre)
4. `road_event_summary` — 15-min window: event counts from telemetry road_event column

From silver/weather: 5. `weather_hourly` — 1-hour window: temperature, humidity, speed_factor (how much weather slows vehicles), heat_category (Cairo context)

From silver/traffic: 6. `traffic_30min` — 30-min window per GPS bucket: congestion ratio, speed deficit vs free-flow, congestion band

From silver/road_events: 7. `incident_summary` — 1-hour window: incident counts with severity scores, GPS coordinates, road type

Why read all 4 Silver tables? Telemetry alone cannot answer: "Did Tahrir Bridge slow down because of traffic or because of a sandstorm?" You need weather + traffic data to attribute the cause. The gold layer enables cross-topic analytical joins in dbt.

alert_consumer.py — Real-Time Alerts (Pure Python)

Why not Spark? Spark was originally used for alerts but consumed 1 executor core from the cluster, causing resource starvation. For 5 vehicles generating 5 messages/second, Python's Kafka consumer is sufficient and uses zero cluster resources. Latency improved from ~5 seconds (Spark micro-batch) to ~1 second (Python poll loop).

Writes to two sinks simultaneously:

1. Kafka `alerts` topic — for downstream consumers
2. PostgreSQL `realtime_alerts` table — for Grafana dashboard queries
3. PostgreSQL `realtime_telemetry` table — for Grafana speed/fuel panels

---

8. STORAGE LAYER (AWS S3 — Medallion Architecture)

Bucket Structure

```
s3://smart-city-datalake/
├── bronze/              ← Raw Parquet, 7-day retention
│   ├── vehicle-telemetry/partition_date=YYYY-MM-DD/partition_hour=H/
│   ├── weather-data/
│   ├── traffic-events/
│   └── road-events/
├── silver/              ← Cleaned Parquet, 30-day retention
│   ├── telemetry/
│   ├── weather/
│   ├── traffic/
│   └── road_events/
├── gold/                ← Aggregated Parquet, 1-year retention
│   ├── vehicle_5min/
```

```
│   ├── route_hourly/
│   ├── fuel_daily/
│   ├── road_event_summary/
│   ├── weather_hourly/
│   ├── traffic_30min/
│   └── incident_summary/
├── _checkpoints/        ← Spark streaming state — NEVER DELETE
└── ml_features/         ← Gold data copied for ML training (future)
```

Why Medallion Architecture?

- **Bronze (raw):** Reprocess Silver from scratch if bugs discovered. Immutable log.
- **Silver (clean):** Validated data usable by multiple consumers (Gold, ML, ad-hoc).
- **Gold (aggregated):** Pre-computed KPIs reduce Snowflake query cost dramatically. Instead of scanning 432,000 telemetry rows for a daily dashboard, query 3 Gold rows.

Partition Strategy

All layers use `partition_date=YYYY-MM-DD/partition_hour=H` (integer hour, no leading zero). This allows Snowflake to use partition pruning — "give me June 5th data" only scans one S3 prefix, not the entire bucket.

Important discovery: Spark writes integer partition columns without leading zeros: `partition_hour=4` NOT `partition_hour=04`. Gold batch reader must use `{hour}` not `{hour:02d}` in the path string — this caused a critical PATH_NOT_FOUND bug that was diagnosed and fixed.

Data Volume Estimates

Layer	Rows/Day	File Size/Day	Cost/Month
Bronze telemetry	~432,000	~50 MB	<$0.01
Silver telemetry	~400,000	~45 MB	<$0.01
Gold vehicle_5min	1,440	<1 MB	negligible
All Gold tables	~5,000	~3 MB	negligible
Total S3 cost	—	—	~$3/month

---

9. ORCHESTRATION LAYER (Apache Airflow)

URL: http://localhost:8082 | **Credentials:** admin/admin

3 DAGs

smart_city_daily_pipeline — runs @hourly

```
get_target_hour (Python)
    → check_silver_exists (ShortCircuit — skips if Silver partition empty)
    → run_gold_batch (BashOperator — spark-submit)
    → check_gold_output (Python — verifies S3 files written)
    → notify_success (Bash — logs completion)
```

Purpose: Automates Gold aggregation without manual intervention. The ShortCircuit is critical —

if Silver hasn't written yet for that hour, Gold batch skips gracefully instead of failing.

smart_city_data_quality — runs every 3 hours Checks Bronze, Silver, Gold layers all have recent data. Logs file counts, sizes, and health score. Catches pipeline stalls before they affect dashboards.

smart_city_s3_lifecycle — runs daily at 2 AM UTC Deletes Bronze partitions older than 7 days. Deletes Silver partitions older than 30 days. Reports S3 bucket usage and estimated cost. Gold is kept 1 year (by S3 bucket lifecycle rules, not Airflow).

---

10. DATA WAREHOUSE LAYER (Snowflake)

Account: cqc62031.us-east-1.snowflakecomputing.com **Warehouse:** SMART_CITY_WH (X-Small, AUTO_SUSPEND=300s) **Database:** SMART_CITY_DB

Three Schemas

- **RAW** — External tables (metadata only, reads S3 directly, zero storage cost)
- **STAGING** — dbt views (zero storage) + dimension tables (~237 rows total)
- **MART** — dbt materialized fact tables (Power BI reads here)

External Tables

Snowflake RAW schema contains 7 external tables pointing directly at S3 Gold Parquet. Zero data is copied into Snowflake. When queried, Snowflake reads the Parquet files from S3 in real time. This means:

- No ETL/ELT loading step needed
- New Gold files are visible immediately after `ALTER EXTERNAL TABLE REFRESH`
- Storage cost = $0 (data stays in S3)

Partition column extracted from S3 path using `SPLIT_PART`:

```
SPLIT PART(SPLIT PART(metadata$filename, 'partition date=', 2), '/', 1)
```

---

11. TRANSFORMATION LAYER (dbt)

Version: dbt-snowflake 1.11.5 on Python 3.11 **Project:** smart_city_dbt **Test results:** 64/64 PASS, WARN=0, ERROR=0

Why dbt when Spark already aggregated?

Spark solves the **scalability problem** — processing 432,000 events efficiently. dbt solves the **semantic layer problem** — what does avg_speed_kmh MEAN to a business user? How do you define "anomaly" consistently across all 4 dashboards? dbt provides version-controlled, tested, documented SQL that creates the dimensional model Power BI depends on.

Model Layers

Staging (7 views — zero storage): Reads RAW external tables. Casts all VARIANT/DOUBLE columns to proper SQL types. Derives categorical columns (speed_band, fuel_band, heat_category,

etc.). No business logic. No joins. Stable interface between RAW and mart layers.

Dimensions (5 tables — static/SCD Type 1): | Table | Type | Rows | Source | |---|---|---|---| | dim_vehicle | SCD Type 1 | 5 | DISTINCT from stg_vehicle_5min | | dim_route | Static | 4 | Hardcoded from cairo_routes.py | | dim_date | Date spine | 214 | Generated (June–Dec 2026) | | dim_weather_condition | Static | 10 | Hardcoded from weather_fetcher.py | | dim_road_event_type | Static | 4 | Hardcoded from vehicle_simulator.py |

dim_date uses Cairo-specific calendar logic:

- Weekend = Friday + Saturday (NOT Saturday + Sunday)
- Rush hours defined as 8-10:30 AM and 16-19:30 PM on workdays

Mart (4 fact tables — materialized): | Table | Grain | Joins | Power BI Dashboard | |---|---|---|---| | mart_vehicle_performance | vehicle × 5min | + dim_vehicle, dim_route, dim_date, weather | Vehicle Tracking | | mart_route_analytics | route × hour | + dim_route, dim_date, weather, traffic | Route Analytics | | mart_fuel_environment | vehicle_type × day | + dim_date, weather_daily | Fuel & Environment | | mart_incidents_safety | event_type × road_type × hour | + dim_road_event_type, dim_date, weather | Incidents & Safety |

dbt Variables (centralized constants)

```
vars:
  fuel_price_egp: 11       # Egypt 95-octane petrol (EGP/litre)
  co2_kg_per_litre: 2.31   # Egypt 95-octane CO2 factor
```

Change once → `dbt run` rebuilds all affected models consistently.

Surrogate Keys

All mart tables have MD5-based surrogate keys:

```
MD5(COALESCE(vehicle_id,'') || '|' || COALESCE(CAST(window_start AS
VARCHAR),''))
```

These give Power BI a single unique identifier per row for relationship management, following Kimball dimensional modeling best practices.

64 Automated Tests (all passing)

- `not_null` tests on all primary/foreign keys
- `unique` tests on all dimension primary keys
- `accepted_values` tests on all categorical columns (speed_band, fuel_band, congestion_level, severity_label, etc.)
- 3 custom singular tests:
  - `assert_no_negative_fuel` — fuel consumption never < 0
  - `assert_valid_gps_bounds` — coordinates within Cairo bounding box
  - `assert_speed_within_limits` — speed within 0–250 km/h

Schema Type: Galaxy Schema

Multiple fact tables sharing some dimensions. Not a pure star (one fact) or snowflake (normalized dimensions). Galaxy because:

- 4 fact tables (vehicle, route, fuel, incidents)
- Shared dimensions: dim_date (used by all 4), dim_route (used by 2)
- Independent dimensions: dim_vehicle, dim_road_event_type, dim_weather_condition

---

12. REAL-TIME ANALYTICS (Grafana + PostgreSQL)

URL: http://localhost:3000 | **Credentials:** admin/smartcity123 **Refresh rate:** 15 seconds

Data Flow

```
Kafka vehicle-telemetry
    → Python alert_consumer.py
    → PostgreSQL realtime_alerts table
    → Grafana (PostgreSQL data source)
    → Dashboard auto-refreshes every 15s
```

PostgreSQL Tables for Grafana

realtime_alerts — one row per alert fired:

```
vehicle_id, alert_type, severity, rule_name, ts,
speed_kmh, engine_temp_c, fuel_pct, rpm,
latitude, longitude, road_type, vehicle_type, route_name
```

realtime_telemetry — one row per 10 vehicle readings:

```
vehicle_id, vehicle_type, route_name, ts,
speed_kmh, engine_temp_c, fuel_level_pct, rpm,
traffic_density, latitude, longitude, road_type, road_event
```

Grafana Dashboard Panels

- Pipeline Architecture (text panel — always visible, no data dependency)
- Prometheus Self Scrape Duration (working — Prometheus UP)
- Real-time alerts table from PostgreSQL
- Vehicle speed timeline from realtime_telemetry

---

13. BATCH ANALYTICS (Power BI)

Connection: Snowflake Import mode **Tables loaded:** 4 mart tables + 5 dimension tables **Refresh:** Manual (run `dbt run` then Power BI Refresh)

4 Dashboards

Dashboard 1 — Fleet Overview: Answers: How is the fleet performing right now (today)? Visuals: KPI cards (5 vehicles, avg speed, anomaly count, fuel level, engine temp), speed

band distribution stacked bar, vehicle by route donut chart, avg speed over time line chart, fuel level by vehicle horizontal bar.

Dashboard 2 — Route Analytics & Congestion: Answers: Which Cairo routes are congested and why? Visuals: Congestion KPI cards, route midpoint scatter map colored by congestion level, 100% stacked bar of speed bands per route, congestion cause pie chart (weather_and_traffic / traffic_volume / weather_conditions / incidents / normal), speed vs time per route line chart.

Dashboard 3 — Fuel & Environment: Answers: Which vehicle type wastes the most fuel? How does Cairo heat affect consumption? Visuals: Total CO2 kg card, fleet cost EGP card, fuel share donut by vehicle type, fuel rate vs efficiency scatter (X=weather_temp_c, Y=avg_fuel_rate_l100km), daily CO2 trend line chart, vehicle type comparison table.

Dashboard 4 — Incidents & Safety: Answers: Where do accidents happen? Which road type is most dangerous? Visuals: Total incidents card, critical incidents card, incident map (GPS coordinates), incidents by type clustered column, incidents by road type stacked bar, incidents over time line chart, recent critical incidents table.

Power BI Relationship Model

DIM_VEHICLE	(1) ⟷ (many)	MART_VEHICLE_PERFORMANCE	[vehicle_id]
DIM_ROUTE	(1) ⟷ (many)	MART_VEHICLE_PERFORMANCE	[route_name]
DIM_ROUTE	(1) ⟷ (many)	MART_ROUTE_ANALYTICS	[route_name]
DIM_DATE	(1) ⟷ (many)	MART_VEHICLE_PERFORMANCE	[partition_date]
DIM_DATE	(1) ⟷ (many)	MART_ROUTE_ANALYTICS	[partition_date]
DIM_DATE	(1) ⟷ (many)	MART_FUEL_ENVIRONMENT	[partition_date]
DIM_DATE	(1) ⟷ (many)	MART_INCIDENTS_SAFETY	[partition_date]
DIM_ROAD_EVENT	(1) ⟷ (many)	MART_INCIDENTS_SAFETY	[event_type]

All cardinality: Many-to-one. All filter direction: Single.

---

14. MACHINE LEARNING PATH

The Silver layer Parquet files in S3 serve as the feature store for ML models:

Implemented Models:

1. **Incident Prediction** — Predict incident/anomaly probability on route segments
    - Features: congestion_ratio, visibility_km, speed, temperature, weather_risk, active_events
    - Target: incident_count > 0 (binary classification with XGBoost)
2. **Vehicle Health Prediction** — Predict vehicle performance issues from telemetry
    - Features: engine_temp, RPM, acceleration, fuel_level
    - Target: vehicle_health_score (regression)
3. **Recommendation Engine** — Generate actionable recommendations
    - Resource Deployment, Route Diversion, Maintenance Alerts, Weather Warnings

S3 path for ML: s3://smart-city-datalake/silver/ — directly readable by scikit-learn/XGBoost via boto3

---

15. DATA QUALITY & TESTING

Pipeline-Level Checks

- **Bronze:** `failOnDataLoss=false` — logs offset gaps, does not crash
- **Silver:** Mandatory null checks + sensor clamping + `had_sensor_clamp` flag
- **Gold:** Batch reader returns empty DataFrame (not None) for missing partitions
- **Airflow:** `check_silver_exists` ShortCircuit before every Gold run

dbt Test Results: 64/64 PASS

Categories:

- 22 `accepted_values` tests — categorical columns only contain valid values
- 19 `not_null` tests — mandatory columns are always populated
- 5 `unique` tests — dimension primary keys have no duplicates
- 3 custom singular tests — domain-specific business rules
- 15 other schema-level tests

Key Data Quality Discoveries During Development

1. Old Silver files (June 2nd) had `DOUBLE` type for speed_kmh while new schema expected `FloatType`. Spark strict vectorized reader refused to convert. Solution: removed schema enforcement from batch reader, used schema inference.
2. Bronze writer used Kafka group.id per topic — warning logged but not an error. Multiple queries with same group.id can interfere on restart. Acceptable for graduation project; production fix would use separate consumer groups.

---

16. TECHNICAL CHALLENGES & SOLUTIONS

Challenge	Root Cause	Solution
kafka-python crashes on Python 3.12+	Library incompatibility	Replaced with kafka-python-ng
Silver job always WAITING when Bronze runs	Bronze grabbed all 4 executor cores	Added spark.cores.max=2 for Bronze, 1 for Silver
All jobs terminated when 3rd job added	Driver processes compete for master container RAM (768m limit)	Moved Alerts to Python on Windows host (0 Spark resources)
kafka_timestamp schema mismatch	Bronze writes TimestampType, Silver schema declared LongType	Fixed Silver schema — both must be TimestampType
Silver "Files were deleted or changed" error	Bronze writes partitioned files — Spark streaming treats as changes	Added ignoreChanges=true to Bronze reader in Silver

Challenge	Root Cause	Solution
Gold batch PATH_NOT_FOUND for all Silver partitions	Spark writes partition_hour=4 (int, no leading zero), batch reader used {hour:02d} = "04"	Changed format string from {hour:02d} to {hour}
Type mismatch reading old Silver Parquet	Old files had DOUBLE, new schema enforced FloatType	Removed .schema() from batch reader, used mergeSchema=true
Power BI OLE DB 0x80040E4E error	Snowflake warehouse auto-suspended between table loads	Increased AUTO_SUSPEND from 60s to 300s, warm up warehouse before refresh
Power BI duplicate value in relationship	Mart tables set as "one" side of relationship — they have many rows per vehicle	Corrected relationship: dim tables are "one" side, mart tables are "many"
dbt Python 3.14 incompatibility	mashumaro 3.14 has a bug on Python 3.14	Created separate dbt_venv with Python 3.11
Grafana No Data for real-time path	Grafana cannot read from Kafka directly	Added PostgreSQL sink to alert_consumer.py, added Grafana PG data source
Gold aggregator only used telemetry	Original script hardcoded silver/telemetry path	Rebuilt gold_aggregator.py v2 to read all 4 Silver sources

---

17. HOW TO RUN THE PROJECT

Prerequisites

- Docker Desktop running
- Python venv activated (`.\venv\Scripts\Activate`)
- dbt venv available (`.\dbt_venv\Scripts\Activate`)
- AWS credentials in `.env` file
- Snowflake account active

Daily Operation (3 terminals)

Terminal 1 — Bronze (keep open):

```
cd "D:\ITI campaign\ITI content\final_project\data_sources\test"
.\spark_jobs\submit_jobs.ps1 -Job bronze
```

Wait 2 minutes for first batch.

Terminal 2 — Silver (keep open):

```
.\spark_jobs\submit_jobs.ps1 -Job silver
```

Wait 1 minute.

Terminal 3 — Real-Time Alerts (keep open):

```
.\venv\Scripts\Activate
cd simulator
```

```
python alert_consumer.py
```

Airflow handles Gold automatically every hour. Monitor at http://localhost:8082

Generate 30 Days of Historical Data

```
# One-time command, ~10 minutes
python simulator\generate_historical_gold.py --days 30

# Then refresh Snowflake external tables
# In Snowflake UI:
ALTER EXTERNAL TABLE SMART_CITY_DB.RAW.vehicle_5min REFRESH;
ALTER EXTERNAL TABLE SMART_CITY_DB.RAW.route_hourly REFRESH;
-- (repeat for all 7 tables)

# Then rebuild dbt mart tables
cd smart_city_dbt
.\..\..\dbt_venv\Scripts\Activate
dbt run

# Then refresh Power BI
```

URLs Quick Reference

Service	URL	Credentials
Kafka UI	http://localhost:8080	none
Spark Master	http://localhost:8081	none
Airflow	http://localhost:8082	admin/admin
Prometheus	http://localhost:9090	none
Grafana	http://localhost:3000	admin/smartcity123
Snowflake	https://cqc62031.us-east-1.snowflakecomputing.com	smartcity/***
dbt docs	http://localhost:8080 (dbt docs serve)	none
Streamlit	http://localhost:8501	none